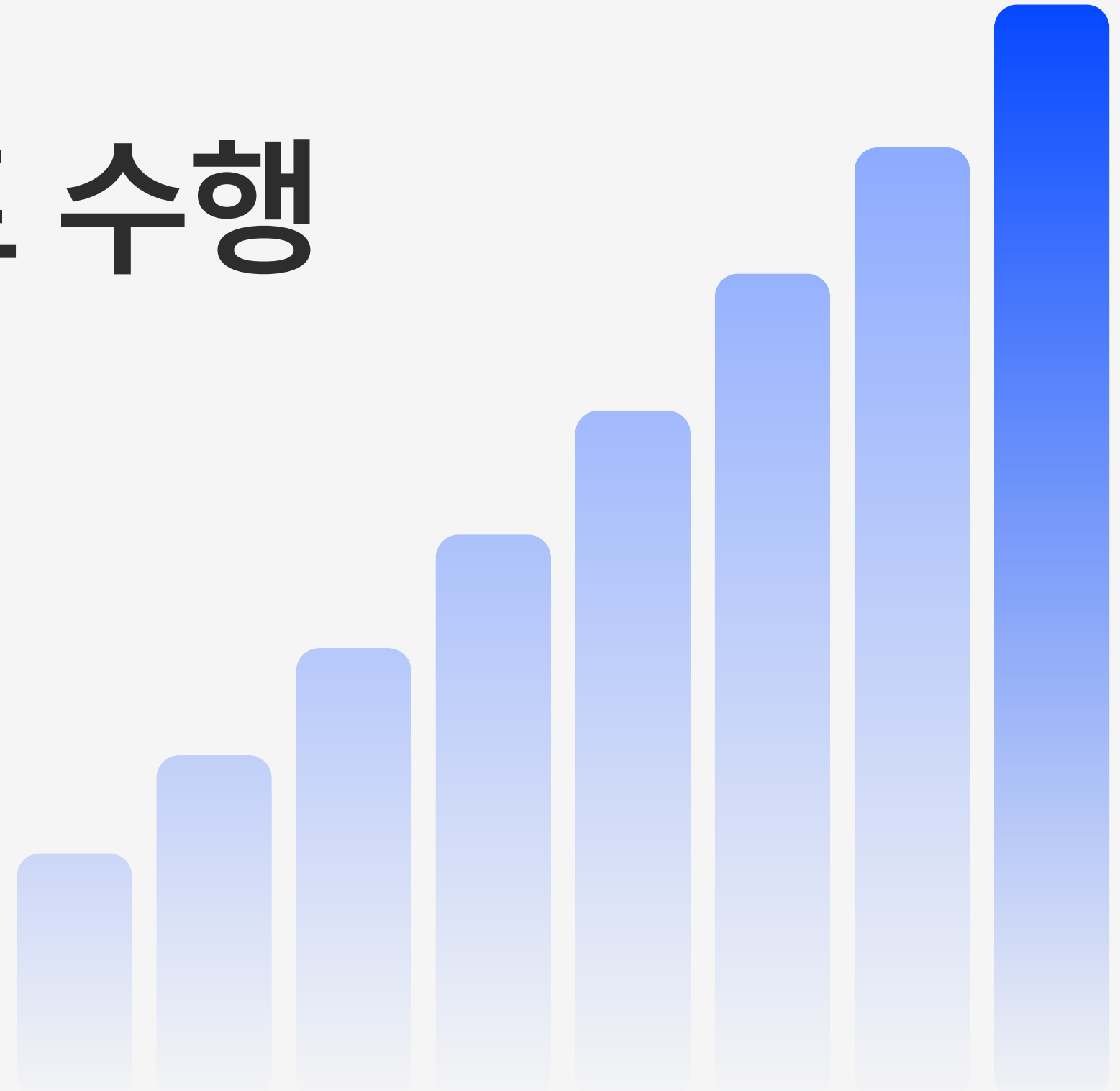


공공데이터 융합 풀스택 개발자 양성과정

애플리케이션 테스트 수행



목 차

01

BookControllerTest 박형규

02

BookRepositoryTest 정승원

03

LendServiceTest 강현욱

04

LibraryIntegrationTest 김정희

05

결과 화면

01 | 코드구현 - BookControllerTest 박형규

```
@WebMvcTest(BookController.class) // BookController 웹 계층만 로드하는 슬라이스 테스트 3개 사용 위치 3 parkhyeonggyu15 +1
class BookControllerTest {

    @Autowired
    private MockMvc mockMvc; // 컨트롤러 요청/응답을 시뮬레이션하는 객체

    @MockitoBean
    private LibraryService libraryService; // LibraryService를 가짜(Mock)로 대체

    private ResultActions postLend(String json) throws Exception {
        // 테스트 클래스 내부에서만 사용하는 private 헬퍼 메서드
        // 반복되는 MockMvc POST 요청 코드를 재사용하기 위해 분리
        // 반환 타입 ResultActions: MockMvc 수행 결과에 대해 andExpect(), andDo() 등을 체이닝할 수 있게 해줌

        return mockMvc.perform(post(uriTemplate: "/api/library/lend")
            // MockMvc를 통해 실제 서버 기동 없이 "/api/library/lend" 경로로 HTTP POST 요청을 시뮬레이션
            // (스프링 시큐리티, 컨트롤러, 필터 등을 포함한 요청/응답 흐름을 애플리케이션 컨텍스트 내에서 테스트)

            .contentType(MediaType.APPLICATION_JSON)
            // 요청 헤더의 Content-Type을 "application/json"으로 설정
            // 컨트롤러가 @RequestBody로 JSON을 파싱할 수 있도록 명시

            .content(json));
        // 요청 바디에 파라미터로 전달받은 JSON 문자열을 그대로 담아 전송
    }
}
```

@WebMvcTest(BookController.class): 웹 계층(컨트롤러)만 로드하는 테스트 설정.

@Autowired MockMvc mockMvc: 실제 서버 구동 없이 HTTP 요청/응답을 시뮬레이션하는 객체.

@MockitoBean LibraryService libraryService: 컨트롤러가 의존하는 LibraryService를 가짜(Mock) 객체로 대체해서, 서비스 로직 없이 컨트롤러 동작만 검증할 수 있게 합니다.

postLend(String json): 반복되는 POST 요청 코드를 재사용하기 위한 private 헬퍼 메서드.

mockMvc.perform(post("/api/library/lend")):

/api/library/lend 경로로 HTTP POST 요청을 시뮬레이션.

contentType(MediaType.APPLICATION_JSON): 요청 헤더의 Content-Type을 JSON으로 설정.

.content(json): 전달받은 JSON 문자열을 요청 바디에 담아 전송.

반환 타입이 ResultActions라서 호출부에서 andExpect(), andDo() 등을 체이닝해 응답 검증 가능.

01 | 코드구현 - BookControllerTest 박형규

```
@Test @ parkhyeonggyu15 +1
@DisplayName("대출 정상: 200 과 lendId 를 반환한다")
// 테스트 목적을 설명하는 표시용 이름 (테스트 리포트에 표시됨)
void lend_정상_200() throws Exception { // 테스트 메서드, MockMvc 등에서 체크된 예외(IOException 등)를 던질 수 있어 throws Exception 선언
    Lend stub = new Lend(); // 서비스 계층이 반환할 가짜(스텝) 결과 객체 생성
    stub.setId(10L); // 스텝 객체의 대출 ID를 10으로 설정 (검증할 때 사용할 값)
    stub.setDueAt(LocalDate.now().plusDays(14)); // 반납 예정일을 현재로부터 14일 뒤로 설정

    when(libraryService.lend(eq(value: 1L), anyString())).thenReturn(stub);
    // libraryService.lend() 메서드가 첫 번째 인자로 정확히 1L, 두 번째 인자로 임의의 문자열이 들어올 때
    // 위에서 만든 stub 객체를 반환하도록 mock(mock) 객체의 동작을 정의

    postLend(json: "{\"bookId\":1,\"member\":\"kim\"}")
    // 대출 요청 API를 호출하는 커스텀 헬퍼 메서드 (내부적으로 MockMvc의 POST 요청을 수행할 것으로 추정)
    // JSON 바디로 bookId=1, member="kim" 을 전달

    .andExpect(status().isOk())
    // HTTP 응답 상태 코드가 200(OK)인지 검증

    .andExpect(jsonPath(expression: "$.lendId").value(expectedValue: 10));
    // 응답 JSON 바디의 "lendId" 필드 값이 10인지 검증 (stub.setId(10L)과 매칭되는지 확인)
}
```

1.Stub 준비

Lend stub = new Lend(): 서비스 계층이 반환할 가짜 결과 객체 생성

stub.setId(10L): 대출 ID를 10으로 설정 (검증 시 사용할 값)

stub.setDueAt(...): 반납 예정일을 현재로부터 14일 뒤로 설정

2.Mock 동작 정의

when(libraryService.lend(eq(1L), anyString())).thenReturn(stub)

libraryService.lend() 메서드가 첫 번째 인자로 정확히 1L, 두 번째 인자로

아무 문자열이나 들어올 때 위에서 만든 stub을 반환하도록 mock(mock) 객체

의 동작을 정의

3.API 호출

postLend("{\"bookId\":1,\"member\":\"kim\"}")

JSON 바디로 bookId=1, member="kim"을 담아 대출 요청 API 호출

4.검증 (assertions)

.andExpect(status().isOk()): HTTP 응답 상태 코드가 200(OK)인지 검증

.andExpect(jsonPath("\$.lendId").value(10)): 응답 JSON 바디의 lendId 필드 값이 10인지 검증 (stub.setId(10L)과 매칭되는지 확인)

01 | 코드구현 - BookControllerTest 박형규

```
@Test ③ parkhyeonggyu15 +1
@DisplayName("대출 검증 실패: bookId 누락/member 공백이면 400")
// 테스트 리포트에 표시될 이름: bookId가 없거나(null) member가 빈 문자열일 때 400을 반환해야 함을 설명
void lend_검증실패_400() throws Exception {
    // 테스트 메서드, MockMvc 수행 중 발생할 수 있는 체크 예외를 던지기 위해 throws Exception 선언

    postLend(json: "{\"bookId\":null,\"member\":\"\"}")
        // 대출 요청 API 호출. bookId를 null로, member를 빈 문자열("")로 채운 잘못된 요청 바디 전송
        // (Bean Validation 등의 @NotNull, @NotBlank 검증에 걸리도록 의도된 입력값)

        .andExpect(status().isBadRequest());
    // HTTP 응답 상태 코드가 400(Bad Request)인지 검증
    // 컨트롤러/DTO에 설정된 검증 어노테이션이 정상 동작하며
    // 잘못된 입력을 걸러내는지 확인하는 테스트
}
```

1.API 호출

`postLend("{\"bookId\":null,\"member\":\"\"}")`

bookId를 null로, member를 빈 문자열("")로 채운 잘못된 요청 바디 전송.

@NotNull, @NotBlank 같은 Bean Validation 검증에 걸리도록 의도된
입력값

2.검증 (assertion)

`.andExpect(status().isBadRequest())`

HTTP 응답 상태 코드가 400(Bad Request)인지 검증

01 | 코드구현 - BookControllerTest 박형규

```
@Test ③ parkhyeonggyu15 +1
@DisplayName("대출 비즈니스 예외: 재고 없음이면 400 + error 메시지")
// 테스트 리포트 표시 이름: 재고가 없는 상황(비즈니스 예외)에서 400과 error 메시지를 반환해야 함을 설명
void lend_재고없음_400() throws Exception {
    // 테스트 메서드, MockMvc 수행 중 발생할 수 있는 체크 예외를 던지기 위해 throws Exception 선언

    when(libraryService.lend(eq(value: 2L), anyString()))
        .thenReturn(new BizException("대출 가능한 재고가 없습니다."));
    // libraryService.lend()가 첫 번째 인자로 정확히 2L, 두 번째 인자로 임의의 문자열이 들어올 때
    // 정상 반환 대신 BizException(비즈니스 예외)을 던지도록 mock 객체의 동작을 정의
    // (재고 부족 등 도메인 규칙 위반 상황을 시뮬레이션)

    postLend(json: "{\"bookId\":\"2\",\"member\":\"kim\"}")
        // 대출 요청 API 호출. bookId=2, member="kim"으로 요청 바디 전송
        // (앞서 스테빙한 조건과 일치시켜 BizException이 발생하도록 유도)

        .andExpect(status().isBadRequest())
        // HTTP 응답 상태 코드가 400(Bad Request)인지 검증
        // 서비스에서 던진 BizException을 컨트롤러/전역 예외 핸들러(@ExceptionHandler 등)가
        // 잡아서 400으로 변환하는지 확인

        .andExpect(jsonPath(expression: "$.error").exists());
    // 응답 JSON 바디에 "error" 필드가 존재하는지 검증
    // 예외 메시지("대출 가능한 재고가 없습니다.")가 error 필드 등을 통해
    // 클라이언트에게 전달되는지 확인 (단, 값 자체는 검증하지 않고 존재 여부만 확인)
}
```

1.Mock 동작 정의

when(libraryService.lend(eq(2L), anyString())).thenReturn(new BizException("대출 가능한 재고가 없습니다."))

libraryService.lend()가 첫 번째 인자로 2L, 두 번째 인자로 임의의 문자열이 들어올 때, 정상 반환 대신 BizException(비즈니스 예외)을 던지도록 mock 설정
재고 부족 등 도메인 규칙 위반 상황을 시뮬레이션

2.API 호출

postLend("{\"bookId\":\"2\",\"member\":\"kim\"}")

bookId=2, member="kim"으로 요청 바디 전송 (앞서 스테빙한 조건과 일치시켜 예외 발생 유도)

3.검증 (assertions)

.andExpect(status().isBadRequest())

HTTP 상태 코드가 400인지 검증

서비스에서 던진 BizException을 컨트롤러/전역 예외 핸들러 (@ExceptionHandler 등)가 400으로 변환하는지 확인

.andExpect(jsonPath("\$.error").exists())

응답 JSON 바디에 error 필드가 존재하는지 검증

예외 메시지("대출 가능한 재고가 없습니다.")가 error 필드 등을 통해 클라이언트에게 전달되는지 확인 (단, 값 자체는 검증하지 않고 존재 여부만 확인)

02 | 코드구현 - BookRepositoryTest 정승원

```
@Autowired // 자동연결
private BookRepository bookRepository; // 테스트할 BookRepository 불러오기

@Test // 테스트 메서드 지정 @js340866 +1
@DisplayName("ISBN 으로 도서를 조회하고 존재 여부를 확인한다") // 테스트 이름을 지정
void findByIsbn_existsByIsbn() {
    Book seed = new Book(); // Book 엔티티 seed 생성
    seed.setTitle("리팩터링"); // seed.title에 "리팩터링" 삽입
    seed.setIsbn("REPO-1"); // seed.Isbn 에 "REPO-1" 삽입
    seed.setTotalCopies(2); // seed.totalCopies 에 정수 2 삽입
    seed.setAvailableCopies(2); // seed.availableCopies 에 정수 2 삽입
    bookRepository.save(seed); // 작성한 엔티티 db에 저장

    Optional<Book> hit = bookRepository.findByIsbn("REPO-1"); // db에서 Isbn = "REPO-1" 인 행이 있는지 찾아 hit에 담는다

    assertAll( // 여러 검증을 모두 실행하고 한 번에 결과를 확인
        () -> assertTrue(hit.isPresent()), // hit 에 값이 존재하는가, 있다면 통과
        () -> assertEquals(expected: "리팩터링", hit.orElseThrow().getTitle()), // hit.title의 값이 "리팩터링" 과 같은가, 같다면 통과
        () -> assertTrue(bookRepository.existsByIsbn("REPO-1")), // db에 저장된 Isbn의 값 중에 "REPO-1" 이 존재하는가, 있다면 통과
        () -> assertFalse(bookRepository.existsByIsbn("NONE-0")) // db에 저장된 Isbn의 값 중에 "NONE-0" 가 존재하는가, 없다면 통과
    );
}
```

BookRepositoryTest

- BookRepository의 Isbn 기반 조회와 존재 여부 기능이 실제 DB에서 정상 동작하며, 저장된 데이터의 정확도를 검증하는 테스트 입니다.


```
@Autowired // 자동연결
private BookRepository bookRepository; // 테스트할 BookRepository 불러오기

@Test // 테스트 메서드 지정 @jsh340866 +1
@DisplayName("ISBN 으로 도서를 조회하고 존재 여부를 확인한다") // 테스트 이름을 지정
void findByIsbn_existsByIsbn() {
    Book seed = new Book(); // Book 엔티티 seed 생성
    seed.setTitle("리팩터링"); // seed.title에 "리팩터링" 삽입
    seed.setIsbn("REPO-1"); // seed.Isbn 에 "REPO-1" 삽입
    seed.setTotalCopies(2); // seed.totalCopies 에 정수 2 삽입
    seed.setAvailableCopies(2); // seed.availableCopies 에 정수 2 삽입
    bookRepository.save(seed); // 작성한 엔티티 db에 저장
```

Book 엔티티 생성

- Book 엔티티를 생성하고 title, isbn, totalCopies, availableCopies 를 설정한 뒤 Repository로 DB에 저장하여 테스트 데이터를 준비하는 과정입니다.

02 | 코드구현 - BookRepositoryTest 정승원

```
Optional<Book> hit = bookRepository.findByIsbn("REPO-1"); // db에서 Isbn = "REPO-1" 인 행이 있는지 찾아 hit에 담는다

assertAll( // 여러 검증을 모두 실행하고 한 번에 결과를 확인
    () -> assertTrue(hit.isPresent()), // hit 에 값이 존재하는가, 있다면 통과
    () -> assertEquals( expected: "리팩터링", hit.orElseThrow().getTitle()), // hit.title의 값이 "리팩터링" 과 같은가, 같다면 통과
    () -> assertTrue(bookRepository.existsByIsbn("REPO-1")), // db에 저장된 Isbn의 값 중에 "REPO-1" 이 존재하는가, 있다면 통과
    () -> assertFalse(bookRepository.existsByIsbn("NONE-0")) // db에 저장된 Isbn의 값 중에 "NONE-0" 가 존재하는가, 없다면 통과
);
```

assertAll()

- Isbn 으로 조회한 결과를 Optional로 받아 존재 여부와 제목을 검증하고, existsByIsbn으로 특정 ISBN 존재 여부(true/false)를 함께 확인하는 테스트입니다.

03 | 코드구현 - LendServiceTest 강현욱

```
// 도서 정상 대출 시 재고 감소 및 반납기한 설정 테스트 추가
@Test @kanghyunuk
@DisplayName("대출 정상: 재고가 1 줄고 반납기한이 설정된다")
void lend_정상() {
    // Given: ID가 1이고 재고가 2권인 도서 데이터와 가짜 리포지토리 행동 정의
    Book book = stockBook(id: 1L, available: 2);
    book.setTitle("클린 코드");
    book.setIsbn("i-1");
    book.setTotalCopies(3);
    when(bookRepository.findById(1L)).thenReturn(Optional.of(book));
    when(lendRepository.save(any(Lend.class))).thenAnswer(InvocationOnMock inv -> inv.getArgument(index: 0));

    // When: 대출 로직 실행
    Lend created = service.lend(bookId: 1L, member: "kim");

    // Then: 결과 검증 (재고가 1권으로 줄고, 반납 기한이 생겼으며, 대출자가 맞는지 확인)
    assertAll(
        () -> assertEquals(expected: 1, book.getAvailableCopies()),
        () -> assertNotNull(created.getDueAt()),
        () -> assertEquals(expected: "kim", created.getMember())
    );
}
```

도서 정상 대출 시 재고 감소 및 반납기한 설정 테스트 추가

- Given: ID가 1이고 재고가 2권인 도서 데이터와 가짜 리포지토리 행동 정의
 - ID가 1L이고 현재 대출 가능한 재고(AvailableCopies)가 2권인 모의 도서(book) 객체를 생성
 - bookRepository.findById(1L)이 호출되면, 위에서 생성한 도서 객체를 반환하도록 설정
 - lendRepository.save(...)가 호출되면, 데이터베이스에 저장되는 대출 기록(Lend) 객체를 그대로 다시 반환(thenAnswer)하도록 함
- When: 대출 로직 실행
 - service.lend(1L, "kim")을 호출하여, ID가 1번인 도서를 "kim"이라는 회원이 대출하는 상황을 실행
 - 호출 결과로 생성된 대출 기록은 created 변수에 할당
- Then: 결과 검증 (재고가 1권으로 줄고, 반납 기한이 생겼으며, 대출자가 맞는지 확인)

03 | 코드구현 - LendServiceTest 강현욱

```
// 대출 경계값(재고 0)일 때 대출 불가 예외 테스트 추가
@Test @kanghyunuk
@DisplayName("대출 경계: 재고 0 이면 대출할 수 없다 (D1 경계값)")
void lend_재고0_예외() {
    // Given: 재고가 0권인 도서 상태 설정
    when(bookRepository.findById(1L)).thenReturn(Optional.of(stockBook(id: 1L, available: 0)));

    // When & Then: 대출 시도 시 BizException 예외가 터지는지 확인
    assertThrows(BizException.class, () -> service.lend(bookId: 1L, member: "kim"));
}
```

```
// 존재하지 않는 도서 ID로 대출 시 예외 발생 테스트 추가
@Test @kanghyunuk
@DisplayName("대출 예외: 존재하지 않는 도서면 BizException (D2)")
void lend_미존재도서_예외() {
    // Given: 존재하지 않는 도서 ID(99번) 조회 시 빈 값(Optional.empty) 반환 설정
    when(bookRepository.findById(99L)).thenReturn(value: Optional.empty());

    // When & Then: 예외가 터지는지 확인하고, 에러 메시지가 존재하는지 검증
    BizException ex = assertThrows(BizException.class, () -> service.lend(bookId: 99L, member: "kim"));
    assertNotNull(ex.getMessage());
}
```

대출 경계값(재고 0)일 때 대출 불가 예외 테스트 추가

- Given: 재고가 0권인 도서 상태 설정
 - bookRepository.findById(1L)이 호출되면, 대출 가능한 재고(AvailableCopies)가 0권인 모의 도서 객체 (stockBook(1L, 0))를 반환하도록 설정

- When & Then: 대출 시도 시 BizException 예외가 터지는지 확인

존재하지 않는 도서 ID로 대출 시 예외 발생 테스트 추가

- Given: 존재하지 않는 도서 ID(99번) 조회 시 빈 값 (Optional.empty) 반환 설정
 - bookRepository.findById(99L)이 호출되면, 도서가 없음을 뜻하는 Optional.empty()를 반환하도록 설정

- When & Then: 예외가 터지는지 확인하고, 에러 메시지가 존재하는지 검증

03 | 코드구현 - LendServiceTest 강현욱

```
// 동일 도서 중복 반납 차단 및 재고 정상 유지 테스트 추가
@Test @kanghyunuk
@DisplayName("반납 예외: 두 번 반납해도 재고는 한 번만 늘고 두 번째는 BizException (03 중복 반납)")
void returnBook_중복반납_차단() {
    // Given: 초기 대출 상태 및 재고가 1권인 도서 설정
    Book book = stockBook(id: 1L, available: 1);
    Lend lend = new Lend();
    lend.setId(1L);
    lend.setBookId(1L);
    when(lendRepository.findById(1L)).thenReturn(Optional.of(lend));
    when(bookRepository.findById(1L)).thenReturn(Optional.of(book));
    when(lendRepository.save(any(Lend.class))).thenAnswer(invocationOnMock inv -> inv.getArgument(index: 0));

    // When: 첫 번째 정상 반납 처리 (재고가 1 -> 2로 증가)
    service.returnBook(lendId: 1L);

    // Then: 동일한 대출 건으로 연속 반납 시 예외가 발생하는지, 재고는 여전히 2권인지 검증
    assertAll(
        () -> assertThrows(BizException.class, () -> service.returnBook(lendId: 1L)),
        () -> assertEquals(expected: 2, book.getAvailableCopies())
    );
}
```

동일 도서 중복 반납 차단 및 재고 정상 유지 테스트 추가

- Given: 초기 대출 상태 및 재고가 1권인 도서 설정
 - lendRepository.findById(1L)과 bookRepository.findById(1L) 호출 시, 각각 준비된 모의 대출 기록과 도서 객체를 반환하도록 설정
 - lendRepository.save() 호출 시, 전달된 대출 기록 객체를 그대로 반환
- When: 첫 번째 정상 반납 처리 (재고가 1 → 2로 증가)
 - service.returnBook(1L)을 호출하여 1번 대출 건에 대한 반납을 처리
- Then: 동일한 대출 건으로 연속 반납 시 예외가 발생하는지, 재고는 여전히 2권인지 검증

03 | 코드구현 - LendServiceTest 강현욱

```
// 반납기한 정각 및 초 단위 연체 기준 경계값 테스트 추가
@Test @DisplayName("연체 경계: 반납기한 당일은 연체가 아니고 그 이후가 연체다 (04 경계값)")
void isOverdue_경계() {
    // Given: 기준 반납 기한 설정
    LocalDateTime due = LocalDateTime.of(year: 2026, month: 1, dayOfMonth: 15, hour: 10, minute: 0);

    // 미반납 상태의 대출 데이터
    Lend open = new Lend();
    open.setDueAt(due);

    // 기한 이틀 전에 이미 반납 완료한 대출 데이터
    Lend done = new Lend();
    done.setDueAt(due);
    done.setReturnedAt(due.minusDays(2));

    // When & Then: 초 단위 경계값 기준에 따른 연체 여부(True/False) 검증
    assertAll(
        () -> assertFalse(service.isOverdue(open, due.minusDays(1))),
        () -> assertFalse(service.isOverdue(open, due)),
        () -> assertTrue(service.isOverdue(open, due.plusSeconds(1))),
        () -> assertFalse(service.isOverdue(done, due.plusDays(10)))
    );
}
```

반납기한 정각 및 초 단위 연체 기준 경계값 테스트 추가

- Given: 기준 반납 기한 설정

- 미반납 상태의 대출 데이터
- 기한 이틀 전에 이미 반납 완료한 대출 데이터
- 2026년 1월 15일 10시 00분 00초를 기준 반납 기한(due)으로 설정
- 반납 기한만 설정되어 있고 아직 반납일(returnedAt)이 비어 있는 대출 객체
- 반납 기한보다 이틀 빠른 시점(due.minusDays(2))에 이미 정상 반납 처리가 완료된 대출 객체

- When & Then: 초 단위 경계값 기준에 따른 연체 여부(True/False) 검증

04 | 코드구현 - LibraryIntegrationTest 김정희

```
//테스트 환경에서 스프링환경과 Bean 들을 주입해서 사용하기 위해 @SpringBootTest 사용 하며
//MVC 컨트롤러를 테스트를 하기위해 MockMvc를 자동으로 생성하기 위해 @AutoConfigureMockMvc 사용
@SpringBootTest
@AutoConfigureMockMvc
class LibraryIntegrationTest {

    //MockMvc, BookRepository 주입
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private BookRepository bookRepository;
```

```
@Test //테스트
@DisplayName("대출하면 재고가 0 이 되고 조회 API 에 반영된다") //테스트할 때 화면에 표시될 이름
void 대출_재고반영_흐름() throws Exception { //모든 예외는 던짐
    Book seed = new Book(); // 책 객체 생성
    seed.setTitle("통합테스트 도서"); //책 이름 설정
    seed.setIsbn("INT-1"); //책 식별기호 설정
    seed.setTotalCopies(1); //책 전체 재고 설정
    seed.setAvailableCopies(1); //책 이용가능 재고 설정
    Long bookId = bookRepository.save(seed).getId(); //생성된 책 객체 DB에 저장후 책 고유번호 Long타입으로 변수에 저장

    String lendJson = "{\"bookId\":\"" + bookId + "\",\"member\":\"hong\"}"; //책 대여 테스트를 위한 JSON형태로 이전 저장한 bookId와 hong이라는 이름으로 String 타입에 저장
```

get,post를 테스트 하기 위해
@SpringBootTest, @AutoConfigureMockMvc
를 사용하고 @Autowired로 주입

통합 테스트
테스트를 할 객체 생성 및
post에서 body에 넣을 JSON 형태의 String 타입 변수 생성

04 | 코드구현 - LibraryIntegrationTest 김정희

```
//perform을 사용하여 Http 요청 실행하며 /api/library/lend로 post 하여 책 대여  
//Json타입으로 주고 받으며 바디에 lendJson을 넣어서 보냄  
//andExpect으로 응답이 200인지 그리고 lendId로 넣은값이 존재하는지 검증  
mockMvc.perform(post(uriTemplate: "/api/library/lend")  
    .contentType(MediaType.APPLICATION_JSON)  
    .content(lendJson))  
    .andExpect(status().isOk())  
    .andExpect(jsonPath(expression: "$.lendId").exists());
```

```
//perform을 사용하여 Http 요청 실행하며 /api/library/books/로 get 하며 경로 주소를 통해 이용가능 한 bookId의 책이 몇권인지 조회  
//andExpect으로 응답이 200인지 그리고 availableCopies의 값이 0인지 검증  
mockMvc.perform(get(uriTemplate: "/api/library/books/" + bookId + "/available"))  
    .andExpect(status().isOk())  
    .andExpect(jsonPath(expression: "$.availableCopies").value(expectedValue: 0));
```

mockMvc의 perform을 통해 http 요청 하며
/api/library/lend로 post 하여 책 대여 테스트
andExpect으로 응답이 200인지 그리고 lendId로 넣은값이
존재하는지 검증

mockMvc의 perform을 통해 http 요청 하며
/api/library/books/로 get 하며 경로 주소를 통해 변수를 넣
어 이용 가능 책 조회 테스트
andExpect으로 응답이 200인지 그리고 availableCopies
의 값이 0인지 검증

BookControllerTest

```

✓ BookControllerTest (com.example.demo) 364ms
  ✓ 대출 검증 실패: bookid 누락/member 공백이 286ms
  ✓ 대출 정상: 200 과 lendId 를 반환한다 68ms
  ✓ 대출 비즈니스 예외: 재고 없음이면 400 + error 10ms

  3 테스트 통과 총 3개의 테스트, 364ms

"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" ...
09:49:36.939 [main] INFO org.springframework.test.context.support.AnnotationConfigCon
09:49:37.128 [main] INFO org.springframework.boot.test.context.SpringBootTest

.   ____          _            __ _ _
/\ \ / ____'_ _ _   _  ( )  __ _ \   \ \
( ( )\___| '_ | '_| | '_ \ V ___|___\ \___\
\ \ / ____| | | | | | | | | | ( ) | | ) ) )
 ' |____|_|_|_| |_| | |_|_|_| \___|_/_/_/_/
=====|_|=====|___/_/_/_/_/

:: Spring Boot ::                (v3.5.5)

```

BookRepositoryTest

```

✓ BookRepositoryTest (com.example.demo) 457ms
  ✓ ISBN 으로 도서를 조회하고 존재 여부를 확인 457ms

  1 테스트 통과 총 1개의 테스트, 457ms

"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" ...
09:50:28.354 [main] INFO org.springframework.test.context.support
09:50:28.516 [main] INFO org.springframework.boot.test.context.Spring

.   ____          _            __ _ _
/\ \ / ____'_ _ _   _  ( )  __ _ \   \ \
( ( )\___| '_ | '_| | '_ \ V ___|___\ \___\
\ \ / ____| | | | | | | | | | ( ) | | ) ) )
 ' |____|_|_|_| |_| | |_|_|_| \___|_/_/_/_/
=====|_|=====|___/_/_/_/_/

:: Spring Boot ::                (v3.5.5)

```

LendServiceTest

LibraryIntegrationTest

✓ LendServiceTest (com.example.demo) 1초 241ms

- ✓ 대출 예외: 존재하지 않는 도서면 BizExcep 1초 212ms
- ✓ 연체 경계: 반납기한 당일은 연체가 아니고 그 0 7ms
- ✓ 대출 정상: 재고가 1 줄고 반납기한이 설정된다 12ms
- ✓ 반납 예외: 두 번 반납해도 재고는 한 번만 늘고 .5ms
- ✓ 대출 경계: 재고 0 이면 대출할 수 없다 (D1 경계 5ms)

✓ 5 테스트 통과 총 5개의 테스트, 1초 241ms

```
"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" ...
```

```
Mockito is currently self-attaching to enable the inline-mo
WARNING: A Java agent has been loaded dynamically (C:\Users
WARNING: If a serviceability tool is in use, please run wit
WARNING: If a serviceability tool is not in use, please run
WARNING: Dynamic loading of agents will be disallowed by de
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only
```

종료 코드 0(으)로 완료된 프로세스

✓ LibraryIntegrationTest (com.example.demo) 608ms

- ✓ 대출하면 재고가 0 이 되고 조회 API 에 반영된다 608ms

✓ 1 테스트 통과 총 1개의 테스트, 608ms

```
C:\Program Files\Java\jdk-21.0.10\bin\java.exe" ...
```

```
09:52:02.016 [main] INFO org.springframework.test.context.
09:52:02.163 [main] INFO org.springframework.boot.test.con
```

```

      .      _      _      _      _
/\ / _ _ ' _ _ _ _ ( _ _ _ _ _ \ \ \ \
( ( ) \ _ _ | ' _ | ' _ | | ' _ \ _ ' | \ \ \ \
\ \ / _ _ _ | | _ | | | | | | | ( | | ) ) )
' _ | _ _ | _ _ | | | | | | | \ _ _ , / / / /
=====|_|=====| _ _ / _ / _ / _ /

```